

DSA 8070 R Session 7: Inference for Covariance Matrix

Whitney Huang, Clemson University

Contents

Introduction	1
Simulating data	2
Sample Covariance and Correlation	2
Wishart Distribution	3
Properties:	3
Statistical Inference	5
Bootstrap Confidence Intervals for Covariance Elements	5
Hypothesis Test for Covariance Matrix	5
High-Dimensional Case: SCM Breakdown	6
Marchenko–Pastur Law and Eigenvalue Dispersion	9
Shrinkage Example: Ledoit-Wolf vs Sample Covariance	11
Shrinkage Estimation (Ledoit-Wolf)	13
Compare Eigenvalues	14
Sparse Estimation (Graphical Lasso)	15
Summary	18

```
# Load packages
library(MASS)           # Multivariate normal simulation
library(corpcor)        # Shrinkage covariance matrix estimation
library(fields)         # Visualization tools for matrices
library(PerformanceAnalytics) # Correlation and covariance visualization
library(ggplot2)        # General data visualization
library(glasso)         # Graphical lasso for sparse precision matrix estimation
library(boot)           # Bootstrap methods for statistical inference
library(Matrix)         # Matrix operations and sparse matrix utilities
```

Introduction

This notebook demonstrates covariance matrix estimation techniques, including:

- Sample covariance matrix
- Wishart distribution and inference
- Shrinkage estimation (Ledoit-Wolf)
- Sparse estimation (Graphical Lasso)

Simulating data

We begin by simulating $n = 100$ observations from a three-dimensional multivariate normal distribution with a specified covariance matrix. This allows us to compare the estimated covariance matrix with the known population covariance matrix.

```
# Simulate multivariate normal data
set.seed(123) # Ensure reproducibility

# Specify the true population covariance matrix
Sigma_true <- matrix(c(1, 0.8, 0.5,
                      0.8, 1, 0.3,
                      0.5, 0.3, 1), ncol = 3)

# Generate n = 100 observations from N_3(0, Sigma_true)
data <- mvrnorm(n = 100, mu = c(0, 0, 0), Sigma = Sigma_true)

# Assign variable names
colnames(data) <- c("X1", "X2", "X3")

# Preview the simulated data
head(data)
```

```
##           X1           X2           X3
## [1,] 0.2405360 -0.76694589 -1.09474911
## [2,] 0.1465984 -0.64944798 -0.08115701
## [3,] 1.4224811  1.53013080  0.88431809
## [4,] 0.2803955  0.06408771 -0.25760205
## [5,] 0.1289004  0.61572797 -0.57887479
## [6,] 1.4773313  1.63774739  1.15650887
```

Sample Covariance and Correlation

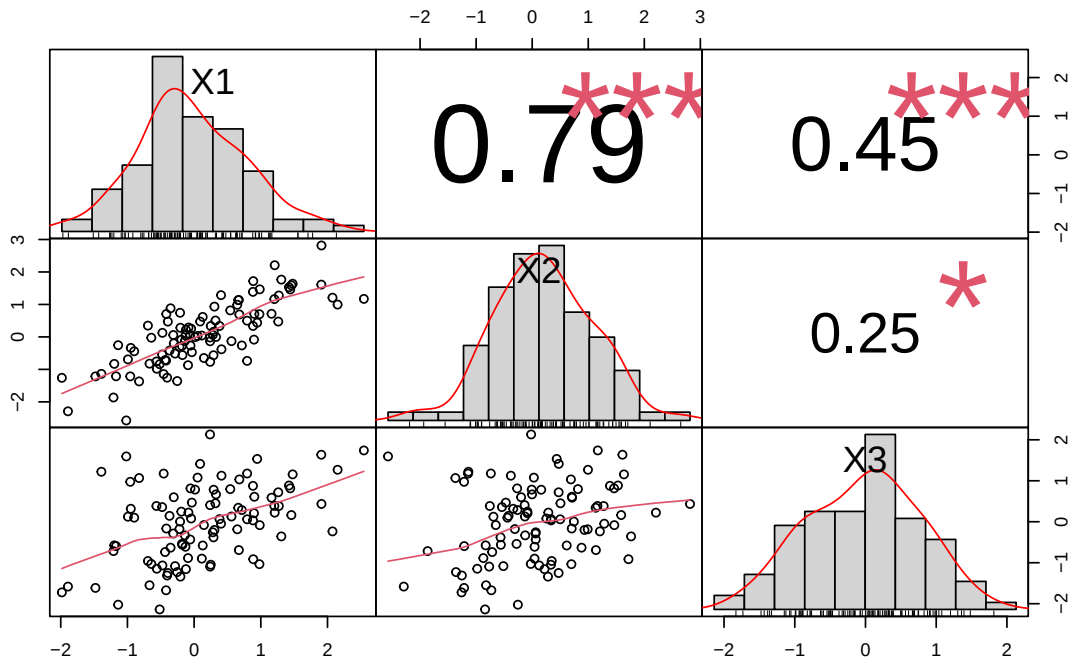
```
# Compute the sample covariance matrix
(S_sample <- cov(data))
```

```
##           X1           X2           X3
## X1 0.7865232 0.6847376 0.3711648
## X2 0.6847376 0.9440958 0.2282530
## X3 0.3711648 0.2282530 0.8557030
```

```
# Compute the sample correlation matrix
(R_sample <- cor(data))
```

```
##           X1           X2           X3
## X1 1.0000000 0.7946216 0.4524280
## X2 0.7946216 1.0000000 0.2539493
## X3 0.4524280 0.2539493 1.0000000
```

```
# Visualize pairwise relationships among variables
# Diagonal: marginal distributions; off-diagonal: scatterplots and correlations
chart.Correlation(data, histogram = TRUE)
```



Wishart Distribution

The *Wishart* distribution is the sampling distribution of the sample covariance matrix of a multivariate normal distribution. If $X_1, \dots, X_n \sim N_p(\mu, \Sigma)$, then:

$$S = \sum_{i=1}^n (X_i - \bar{X})(X_i - \bar{X})^\top \sim W_p(n-1, \Sigma)$$

Properties:

1. Mean: $\mathbb{E}[S] = (n-1)\Sigma$
2. Variance: $\text{Var}[S_{ij}] = (n-1)[\Sigma_{ij}^2 + \Sigma_{ii}\Sigma_{jj}]$
3. $\hat{\Sigma}_n = \frac{S}{n-1} \sim \frac{W_p(n-1, \Sigma)}{n-1} \Rightarrow$ fundamental for statistical inference on Σ

```
# ---- Grid and Matérn covariance ----
t <- seq(0, 1, 0.1)          # 11 grid points in [0,1]
p <- length(t)               # dimension p = 11

# Wrapper for a Matérn covariance; 'pars' = (variance, range, smoothness)
cov.Matern <- function(h, pars) Matern(h, phi = pars[1], range = pars[2], smoothness = pars[3])

dist <- rdist(t)              # pairwise distances between grid points (11x11)
Sigma_Matern <- cov.Matern(dist, c(1, 0.1, 2)) # true covariance matrix (p x p)

# ---- Wishart samples and sample covariance (n = 100) ----
```

```

n <- 100
# rWishart(m, df, Sigma) returns an array p x p x m of Wishart draws
wishart_samples <- rWishart(100, df = n - 1, Sigma_Matern) # 100 Wishart draws with df=99

set.seed(124) # reproducibility
# Simulate n i.i.d. N_p(0, Sigma_Matern) observations
data <- mvrnorm(n = 100, mu = rep(0, p), Sigma = Sigma_Matern)
S_sample <- cov(data) # sample covariance matrix (SCM) for n=100

# ---- Larger-sample SCM (n = 10,000) for comparison ----
set.seed(124)
data1 <- mvrnorm(n = 10000, mu = rep(0, p), Sigma = Sigma_Matern)
S_sample1 <- cov(data1) # SCM for n=10,000 (closer to truth)

# ---- Visualization setup ----
par(mfrow = c(1, 4), # 1 row, 4 panels
    mar = c(1.5, 1.5, 1.5, 4),
    mgp = c(2, 1, 0), family = "serif")

# Common color scale across panels for fair comparison.
# Wishart(df, Sigma) has E[ W / df ] = Sigma, so we scale by df (=99 here).
zlim <- range(S_sample, wishart_samples[, , 1] / 99)

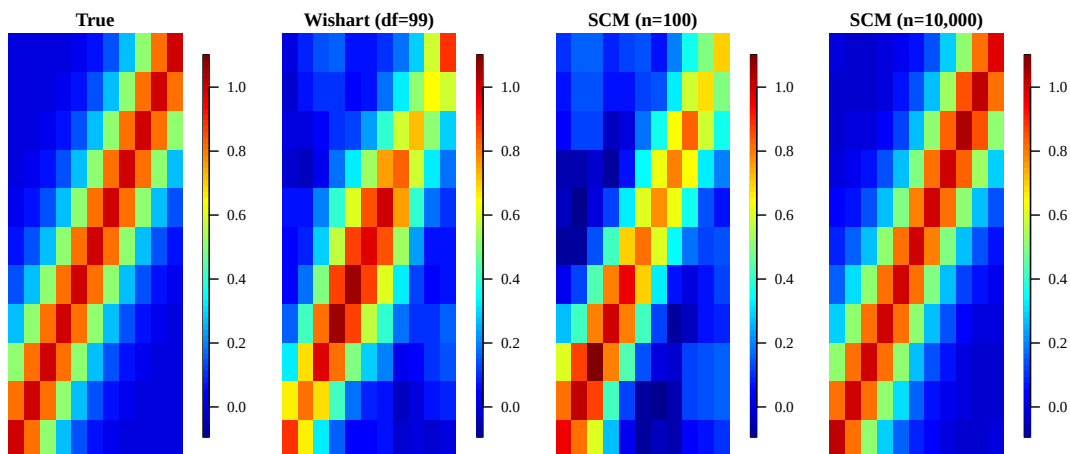
# ---- Panels: true Sigma vs. estimates ----
image.plot(Sigma_Matern, zlim = zlim, axes = FALSE, main = "True",
           legend.mar = 2, legend.cex = 0.5)

image.plot(wishart_samples[, , 1] / 99, axes = FALSE, zlim = zlim,
           legend.mar = 2, main = "Wishart (df=99)", legend.cex = 0.5)
# ^ One Wishart draw scaled by df to be on the Sigma scale.

image.plot(S_sample, zlim = zlim, axes = FALSE, main = "SCM (n=100)",
           legend.mar = 2, legend.cex = 0.5)

image.plot(S_sample1, zlim = zlim, axes = FALSE, legend.mar = 2,
           main = "SCM (n=10,000)", legend.cex = 0.5)

```



To illustrate the sampling variability of the sample covariance matrix (SCM), we consider a Matérn covariance

matrix evaluated at 11 equally spaced locations. Under multivariate normality, the scaled SCM follows a Wishart distribution. The plots compare the true covariance matrix with a scaled Wishart realization and SCMs based on $n = 100$ and $n = 10,000$ observations. As the sample size increases, the SCM becomes more stable and closely resembles the true covariance structure.

Statistical Inference

Bootstrap Confidence Intervals for Covariance Elements

We can use the bootstrap to quantify uncertainty in an individual covariance estimate without relying directly on its asymptotic sampling distribution. Here, we estimate the sampling uncertainty of the covariance between X_1 and X_2 by resampling the observations and constructing a BCa bootstrap confidence interval.

```
# Define the statistic to be estimated in each bootstrap sample
boot_cov <- function(data, indices) {
  # Resample observations using the bootstrap indices
  d <- data[indices, ]
  # Return the sample covariance between X1 and X2
  return(cov(d)[1, 2])
}

# Perform 1,000 bootstrap resamples
set.seed(123)
boot_obj <- boot(data, statistic = boot_cov, R = 1000)

# Construct a bias-corrected and accelerated (BCa) confidence interval
boot.ci(boot_obj, type = "bca")

## BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS
## Based on 1000 bootstrap replicates
##
## CALL :
## boot.ci(boot.out = boot_obj, type = "bca")
##
## Intervals :
## Level      BCa
## 95%      ( 0.5859,  1.1292 )
## Calculations and Intervals on Original Scale
```

Hypothesis Test for Covariance Matrix

We can test whether the population covariance matrix equals a specified matrix. Here, we test $H_0 : \Sigma = I$, corresponding to unit variances and zero covariances. Under multivariate normality, the likelihood ratio test statistic has an asymptotic chi-square distribution with $p(p + 1)/2$ degrees of freedom.

```
# Test H0: Sigma = I versus H1: Sigma != I

# Sample size and dimension
n <- nrow(data)
p <- ncol(data)

# Specify the covariance matrix under H0
```

```

Sigma0 <- diag(p)

# Compute the likelihood ratio test statistic
LRT_stat <- n * (log(det(Sigma0)) - log(det(S_sample)) +
                sum(diag(solve(Sigma0) %*% S_sample)) - p)

# Display the test statistic
LRT_stat

```

```
## [1] 1266.969
```

```

# Under H0, the LRT statistic is asymptotically chi-square
# with p(p + 1)/2 degrees of freedom
df <- p * (p + 1) / 2

# Compute the upper-tail p-value
p_value <- 1 - pchisq(LRT_stat, df)
p_value

```

```
## [1] 0
```

High-Dimensional Case: SCM Breakdown

We next consider a high-dimensional setting with $n = 50$ observations and $p = 100$ variables, where the true covariance matrix is I_{100} . Although the variables are uncorrelated in the population, the SCM can exhibit substantial spurious covariance due to sampling variability. Moreover, because $p > n$, the SCM is rank deficient and therefore singular.

```

# Simulate a high-dimensional setting with n = 50 and p = 100
set.seed(456)

# Generate data from N_p(0, I), so the true covariance matrix is identity
data_hd <- mvrnorm(n = 50, mu = rep(0, 100), Sigma = diag(100))

# Compute the sample covariance matrix (SCM)
S_hd <- cov(data_hd)

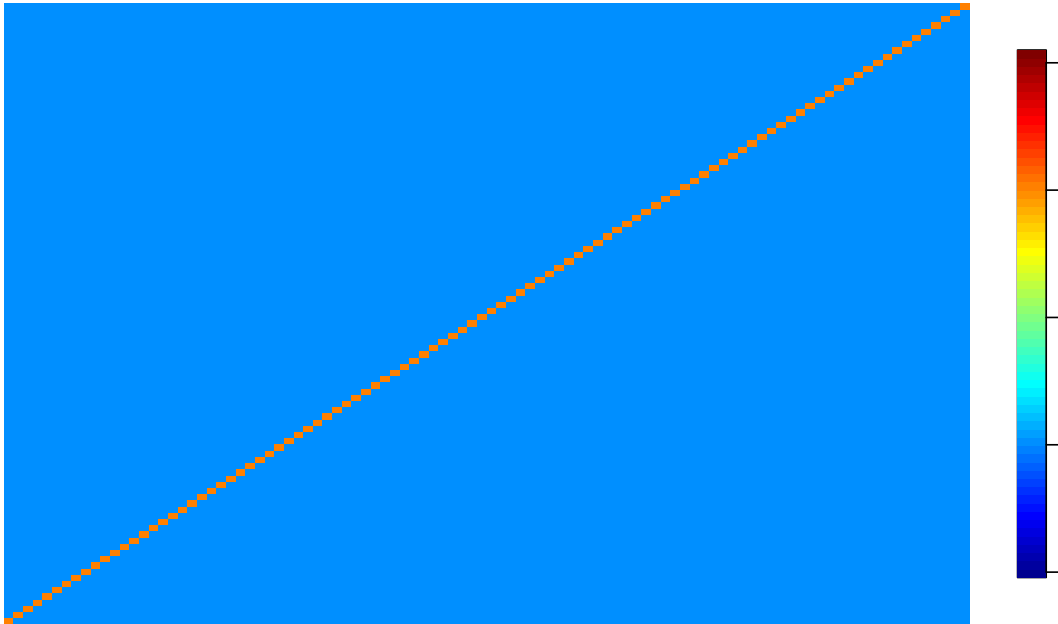
# Set graphical parameters
par(mar = c(1.5, 1.5, 1.5, 2), mgp = c(2, 1, 0), font = 3,
    family = "serif")

# Use a common color scale to compare the true and estimated covariance matrices
zlim <- range(S_hd)

# Visualize the true covariance matrix
image.plot(diag(100), zlim = zlim, axes = FALSE, main = "True",
           legend.mar = 1, legend.cex = 0.25, legend.shrink = 0.85)

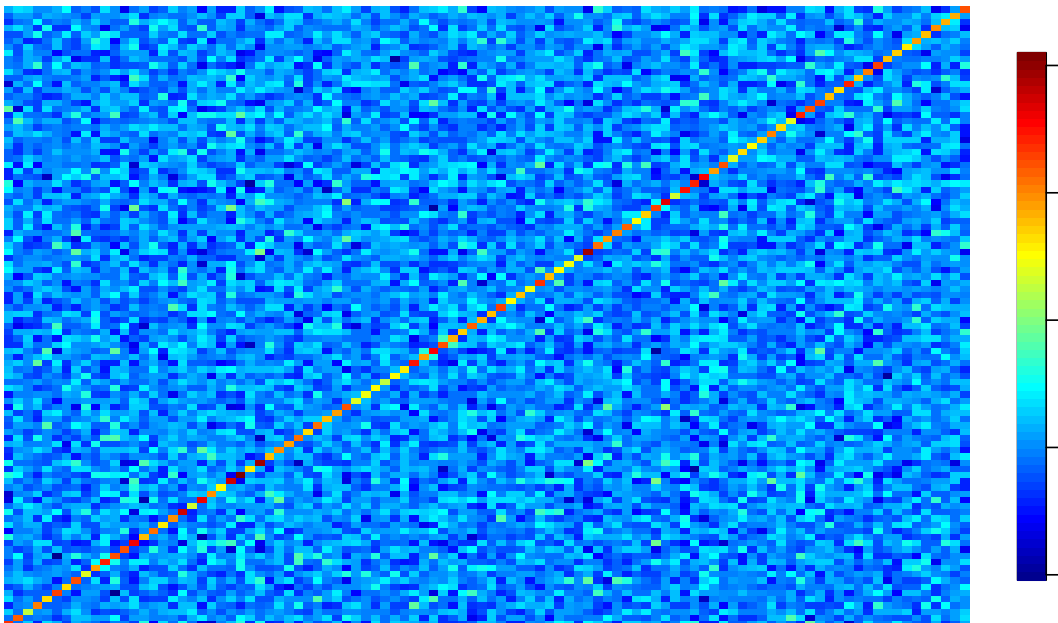
```

True



```
# Visualize the sample covariance matrix
# Off-diagonal patterns arise entirely from sampling variability
image.plot(S_hd, zlim = zlim, axes = FALSE, main = "SCM",
           legend.mar = 1, legend.cex = 0.25, legend.shrink = 0.85)
```

SCM



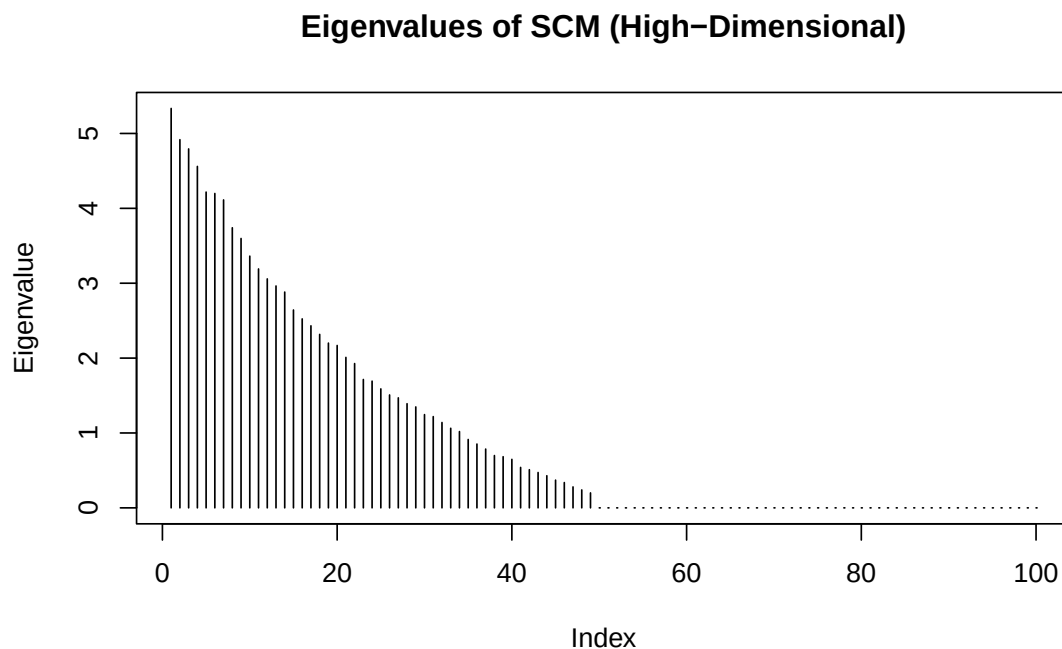
The instability of the SCM can also be seen through its eigenvalues. Although all population eigenvalues are equal to 1 because $\Sigma = I$, the sample eigenvalues are widely dispersed. Since $p > n$, the SCM is rank deficient, resulting in many eigenvalues that are numerically zero. This illustrates that substantial eigenvalue distortion can arise purely from sampling variability in high-dimensional settings.

```
# Compute the eigenvalues of the sample covariance matrix
eigen_hd <- eigen(S_hd)$values
```

```
# Summarize the distribution of the sample eigenvalues
summary(eigen_hd)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.0000  0.0000  0.0000  0.9751  1.5281  5.3342
```

```
# Plot the sample eigenvalues
plot(eigen_hd,
     main = "Eigenvalues of SCM (High-Dimensional)",
     ylab = "Eigenvalue",
     type = "h")
```



```
# Compute sample and true eigenvalues
eig_scm <- eigen(S_hd)$values
eig_true <- eigen(diag(100))$values # All true eigenvalues equal 1
```

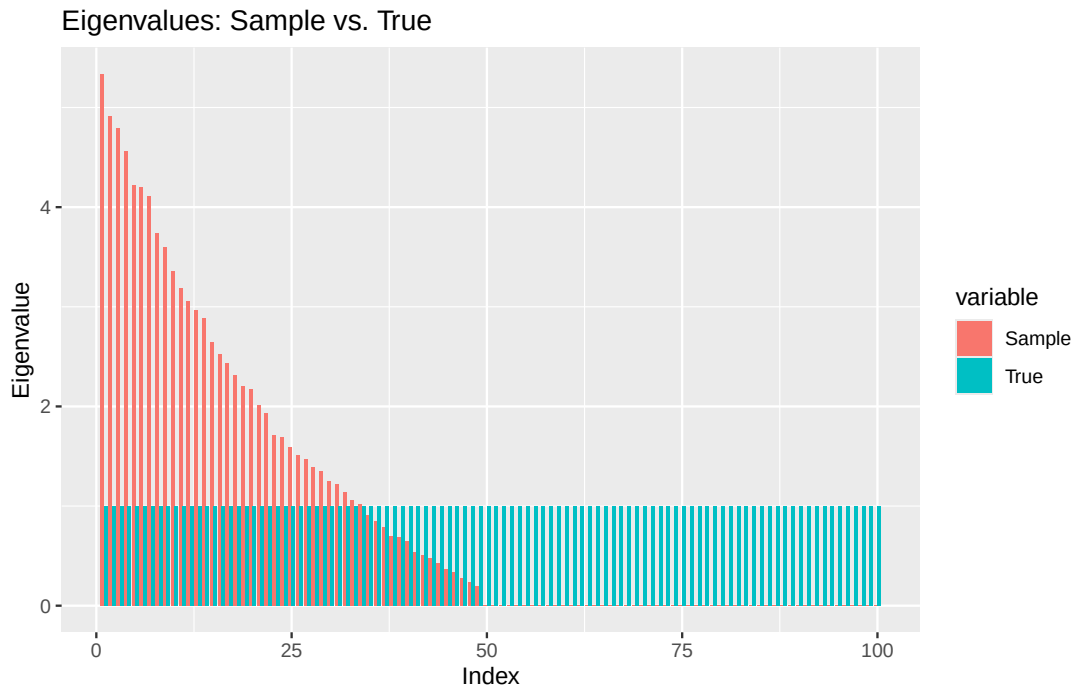
```
# Organize the eigenvalues for comparison
df_eig <- data.frame(
  Index = 1:100,
  Sample = eig_scm,
  True = eig_true
)
```

```
# Reshape the data into long format for ggplot
library(reshape2)
df_melt <- melt(df_eig, id = "Index")
```

```
# Compare the sample eigenvalues with the true eigenvalues
```

```
# Large dispersion in the sample eigenvalues is caused by sampling variability
p <- ggplot(df_melt, aes(x = Index, y = value, fill = variable)) +
  geom_bar(stat = "identity", position = "dodge") +
  labs(title = "Eigenvalues: Sample vs. True",
       y = "Eigenvalue")

print(p)
```



Marchenko–Pastur Law and Eigenvalue Dispersion

To understand why sample eigenvalues become unstable in high dimensions, we simulate data with $\Sigma = I$ and $p/n = 0.5$. Although all population eigenvalues are exactly 1, the sample eigenvalues are spread over a wide range. In high-dimensional settings, this empirical distribution is described by the Marchenko–Pastur law, showing that substantial eigenvalue dispersion can arise purely from sampling variability.

```
# Eigenvalue distribution vs Marchenko-Pastur law
# Compare empirical eigenvalues with the Marchenko-Pastur distribution
set.seed(123)

# Specify the sample size and dimension
n <- 500
p <- 250
c_ratio <- p / n # Dimension-to-sample-size ratio = 0.5

# Generate n observations from N_p(0, I)
X <- matrix(rnorm(n * p), nrow = n, ncol = p)

# Compute the sample covariance matrix using the 1/n scaling
S <- (1 / n) * t(X) %*% X
```

```

# Compute the sample eigenvalues
eigvals <- eigen(S, symmetric = TRUE, only.values = TRUE)$values

# Define the Marchenko-Pastur density
mp_density <- function(x, c) {

  # Lower and upper limits of the MP distribution
  a <- (1 - sqrt(c))^2
  b <- (1 + sqrt(c))^2

  # Density is zero outside the MP support
  dens <- rep(0, length(x))
  inside <- (x >= a & x <= b)

  # Evaluate the MP density within its support
  dens[inside] <- sqrt((b - x[inside]) * (x[inside] - a)) /
    (2 * pi * c * x[inside])

  dens
}

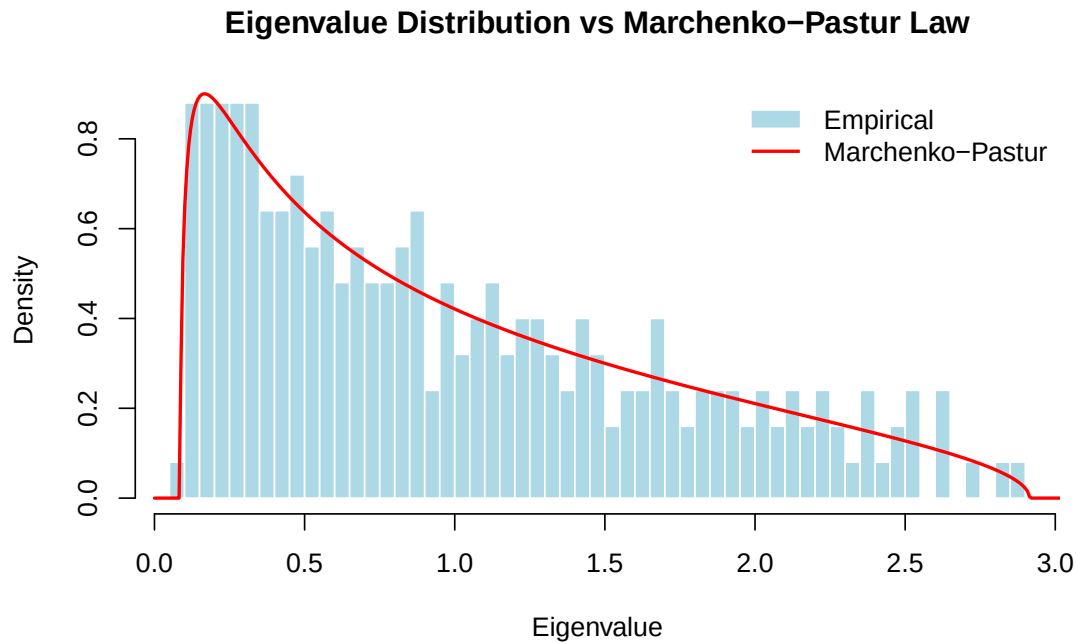
# Evaluate the theoretical MP density
x_grid <- seq(0, max(eigvals) * 1.1, length.out = 500)
mp_dens <- mp_density(x_grid, c_ratio)

# Plot the empirical eigenvalue distribution
hist(eigvals, breaks = 50, freq = FALSE, col = "lightblue",
     border = "white",
     main = "Eigenvalue Distribution vs Marchenko-Pastur Law",
     xlab = "Eigenvalue")

# Overlay the theoretical Marchenko-Pastur density
lines(x_grid, mp_dens, col = "red", lwd = 2)

# Add a legend
legend("topright",
      legend = c("Empirical", "Marchenko-Pastur"),
      col = c("lightblue", "red"),
      lwd = c(10, 2), bty = "n", pch = c(15, NA))

```



Shrinkage Example: Ledoit-Wolf vs Sample Covariance

Ledoit–Wolf Shrinkage Estimation

When p is large relative to n , the sample covariance matrix can be unstable or singular. Shrinkage estimation improves stability by pulling the SCM toward a simpler target. Here, with $p = 50 > n = 30$, we use *Ledoit–Wolf* shrinkage and compare its eigenvalues and numerical conditioning with those of the SCM.

```
# Compare the SCM with Ledoit-Wolf shrinkage
set.seed(123)

# Load package for shrinkage covariance estimation
library(corpcor)

# Specify dimension and sample size
p <- 50
n <- 30

# Generate data from  $N_p(0, I)$ 
X <- matrix(rnorm(n * p), nrow = n, ncol = p)

# Compute the sample covariance matrix (SCM)
S <- cov(X)

# Compute the Ledoit-Wolf shrinkage covariance estimate
S_shrink <- cov.shrink(X)
```

```
## Estimating optimal shrinkage intensity lambda.var (variance vector): 1
##
## Estimating optimal shrinkage intensity lambda (correlation matrix): 0.9931
```

```

# Compute eigenvalues of both covariance estimates
eig_sample <- eigen(S, symmetric = TRUE, only.values = TRUE)$values
eig_shrink <- eigen(S_shrink, symmetric = TRUE, only.values = TRUE)$values

# Compare numerical conditioning
# A smaller condition number indicates a better-conditioned matrix
cond_sample <- max(eig_sample) / min(eig_sample)
cond_shrink <- max(eig_shrink) / min(eig_shrink)

cat("Condition number (sample):", cond_sample, "\n")

```

```
## Condition number (sample): -4.027124e+15
```

```
cat("Condition number (shrinkage):", cond_shrink, "\n")
```

```
## Condition number (shrinkage): 1.031591
```

```

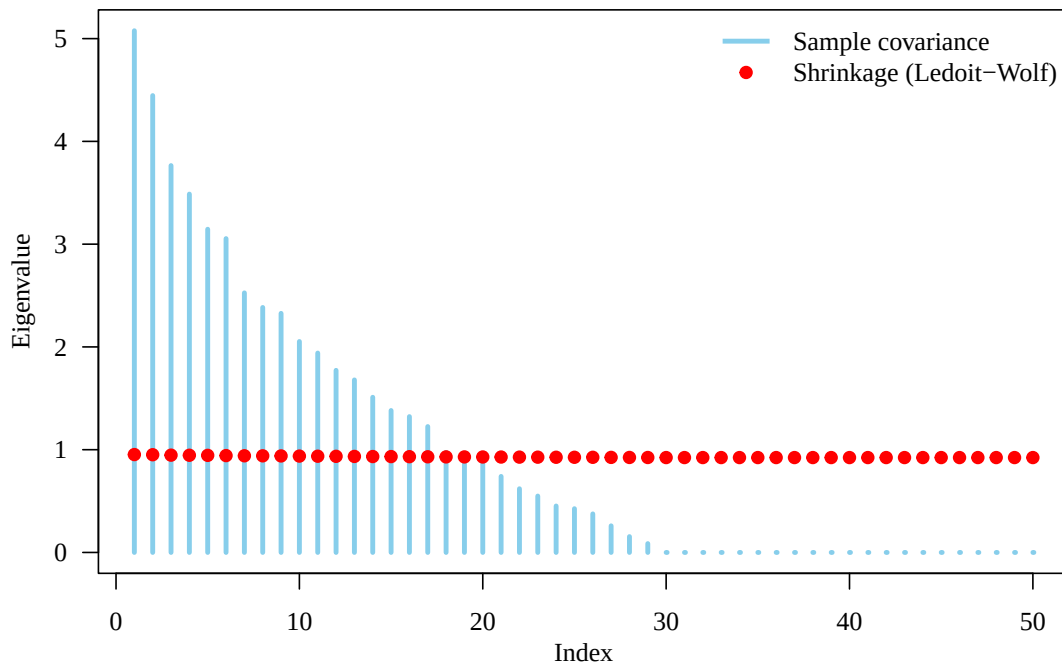
# Compare the eigenvalues of the SCM and shrinkage estimator
par(mar = c(3.5, 3.5, 1, 0.5), mfp = c(2, 1, 0),
    family = "serif", las = 1)

plot(eig_sample, type = "h", lwd = 3, col = "skyblue",
     ylim = c(0, max(c(eig_sample, eig_shrink))),
     xlab = "Index", ylab = "Eigenvalue",
     main = "")

# Add the shrinkage eigenvalues
points(eig_shrink, col = "red", pch = 19)

# Add legend
legend("topright",
     legend = c("Sample covariance", "Shrinkage (Ledoit-Wolf)"),
     col = c("skyblue", "red"),
     lwd = c(3, NA), pch = c(NA, 19), bty = "n")

```



Shrinkage Estimation (Ledoit-Wolf)

```
# Compute a shrinkage covariance estimate using a fixed shrinkage intensity
# lambda = 0.2 shrinks the sample covariance matrix toward the default target
S_shrink <- cov.shrink(data, lambda = 0.2, verbose = TRUE)
```

```
## Estimating optimal shrinkage intensity lambda.var (variance vector): 0.8628
##
## Specified shrinkage intensity lambda (correlation matrix): 0.2
```

```
# Display the shrinkage covariance matrix
S_shrink
```

```
##           [,1]      [,2]      [,3]      [,4]      [,5]      [,6]
## [1,]  0.84714490  0.55832958  0.4018342168  0.19070994  0.022876275 -0.05709991
## [2,]  0.55832958  0.85487403  0.5684408262  0.28926751  0.081807202 -0.04294704
## [3,]  0.40183422  0.56844083  0.8647851336  0.51836983  0.297117763  0.09919590
## [4,]  0.19070994  0.28926751  0.5183698261  0.85238768  0.555992749  0.31056215
## [5,]  0.02287627  0.08180720  0.2971177632  0.55599275  0.845873697  0.53087671
## [6,] -0.05709991 -0.04294704  0.0991958959  0.31056215  0.530876714  0.82612388
## [7,] -0.02862804 -0.06381759  0.0006732619  0.08560092  0.247321428  0.49677857
## [8,] -0.03400519 -0.02861341 -0.0136763742 -0.04315885  0.044459528  0.27598647
## [9,]  0.02932403  0.09070457  0.0887433889 -0.01566876  0.005757439  0.13678750
## [10,] 0.05328648  0.11757578  0.1082462348  0.04944503  0.057607184  0.10759118
## [11,] 0.07820854  0.12295560  0.1171212567  0.07138615  0.102612708  0.12347245
##           [,7]      [,8]      [,9]      [,10]     [,11]
## [1,] -0.0286280402 -0.03400519  0.029324027  0.05328648  0.07820854
## [2,] -0.0638175902 -0.02861341  0.090704565  0.11757578  0.12295560
## [3,]  0.0006732619 -0.01367637  0.088743389  0.10824623  0.11712126
## [4,]  0.0856009235 -0.04315885 -0.015668762  0.04944503  0.07138615
```

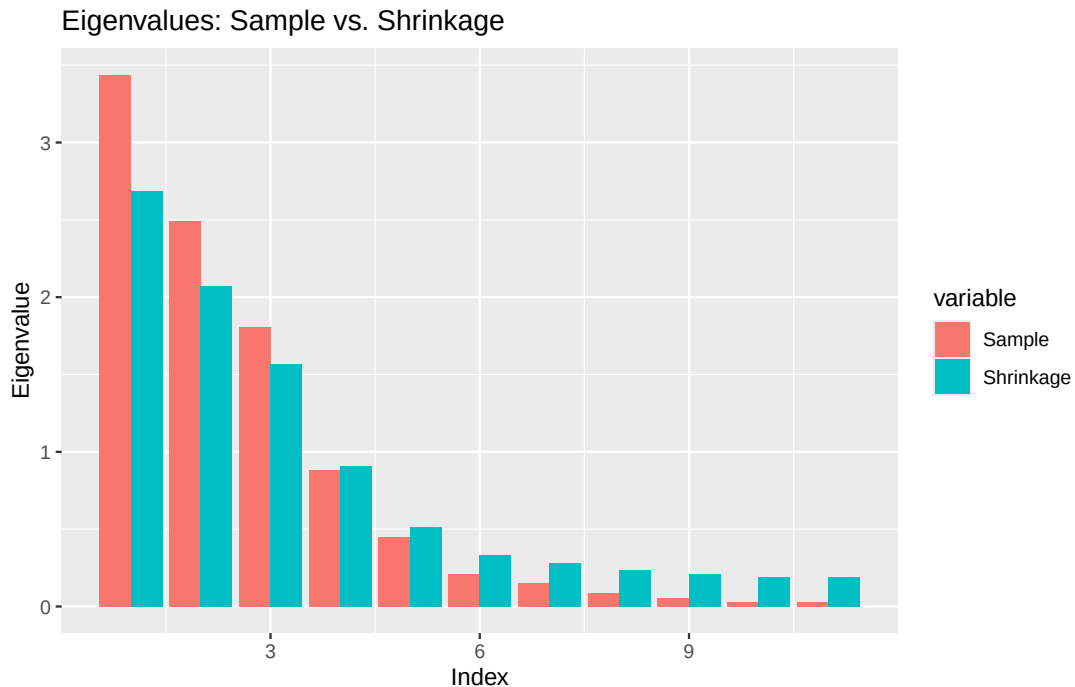
```
## [5,] 0.2473214280 0.04445953 0.005757439 0.05760718 0.10261271
## [6,] 0.4967785659 0.27598647 0.136787498 0.10759118 0.12347245
## [7,] 0.8224395698 0.52631738 0.302628850 0.11688104 0.05746415
## [8,] 0.5263173759 0.82473903 0.519816006 0.28383147 0.17481116
## [9,] 0.3026288504 0.51981601 0.828653385 0.50605525 0.30560032
## [10,] 0.1168810411 0.28383147 0.506055250 0.80786826 0.46873426
## [11,] 0.0574641452 0.17481116 0.305600318 0.46873426 0.81103568
## attr("lambda")
## [1] 0.2
## attr("lambda.estimated")
## [1] FALSE
## attr("class")
## [1] "shrinkage"
## attr("lambda.var")
## [1] 0.862776
## attr("lambda.var.estimated")
## [1] TRUE
```

Compare Eigenvalues

```
eig_sample <- eigen(S_sample)$values
eig_shrink <- eigen(S_shrink)$values
df_eig <- data.frame(Index = 1:11,
                     Sample = eig_sample,
                     Shrinkage = eig_shrink)

library(reshape2)
df_melt <- melt(df_eig, id = "Index")

ggplot(df_melt, aes(x = Index, y = value, fill = variable)) +
  geom_bar(stat = "identity", position = "dodge") +
  labs(title = "Eigenvalues: Sample vs. Shrinkage", y = "Eigenvalue")
```



Sparse Estimation (Graphical Lasso)

Rather than estimating the covariance matrix directly, we can estimate the precision matrix $\Theta = \Sigma^{-1}$. The graphical lasso encourages sparsity in the precision matrix by shrinking some off-diagonal entries toward zero. Zero entries correspond to conditional independence between pairs of variables under a multivariate normal model.

```
# Compute the sample covariance matrix
S <- cov(data)

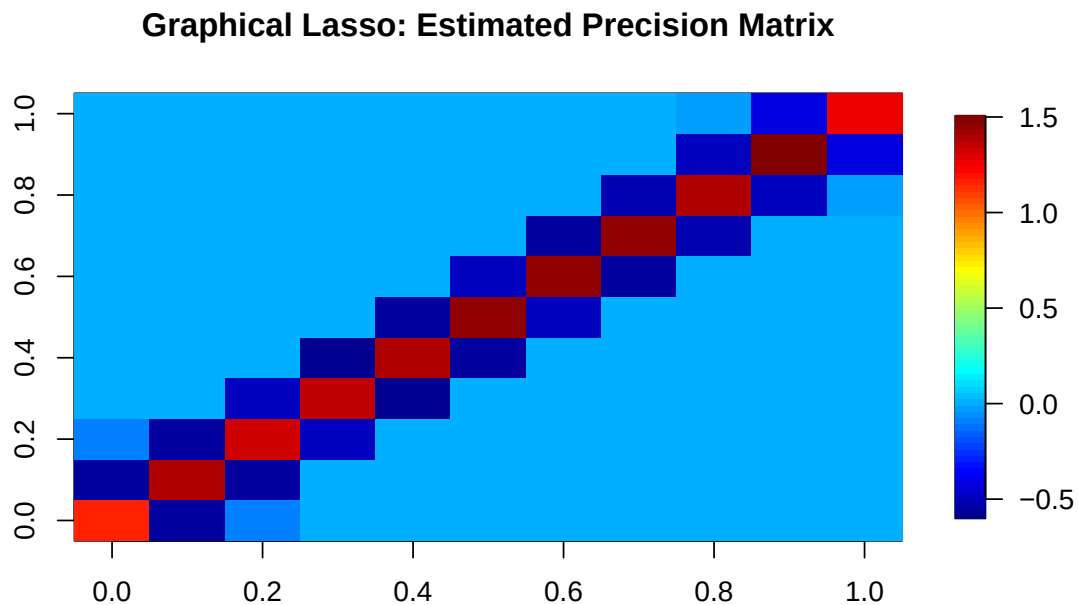
# Fit the graphical lasso with penalty parameter rho = 0.2
# Larger rho produces a sparser estimated precision matrix
S_glasso <- glasso(S, rho = 0.2)

# Extract and display the estimated precision matrix
S_glasso$wi
```

```
##           [,1]      [,2]      [,3]      [,4]      [,5]      [,6]
## [1,]  1.17510177 -0.5453156 -0.08190783  0.0000000  0.0000000  0.0000000
## [2,] -0.54531584  1.4022764 -0.55978201  0.0000000  0.0000000  0.0000000
## [3,] -0.08190066 -0.5597838  1.31369962 -0.4889460  0.0000000  0.0000000
## [4,]  0.00000000  0.0000000 -0.48895530  1.3639429 -0.5845411  0.0000000
## [5,]  0.00000000  0.0000000  0.00000000 -0.5845449  1.4044481 -0.5424634
## [6,]  0.00000000  0.0000000  0.00000000  0.0000000 -0.5424608  1.4475223
## [7,]  0.00000000  0.0000000  0.00000000  0.0000000  0.0000000 -0.4802424
## [8,]  0.00000000  0.0000000  0.00000000  0.0000000  0.0000000  0.0000000
## [9,]  0.00000000  0.0000000  0.00000000  0.0000000  0.0000000  0.0000000
## [10,] 0.00000000  0.0000000  0.00000000  0.0000000  0.0000000  0.0000000
## [11,] 0.00000000  0.0000000  0.00000000  0.0000000  0.0000000  0.0000000
##           [,7]      [,8]      [,9]     [,10]     [,11]
```

```
## [1,] 0.0000000 0.0000000 0.0000000 0.0000000 0.0000000
## [2,] 0.0000000 0.0000000 0.0000000 0.0000000 0.0000000
## [3,] 0.0000000 0.0000000 0.0000000 0.0000000 0.0000000
## [4,] 0.0000000 0.0000000 0.0000000 0.0000000 0.0000000
## [5,] 0.0000000 0.0000000 0.0000000 0.0000000 0.0000000
## [6,] -0.4802518 0.0000000 0.0000000 0.0000000 0.0000000
## [7,] 1.4510863 -0.5429077 0.0000000 0.0000000 0.0000000
## [8,] -0.5429045 1.4663876 -0.52683235 0.0000000 0.0000000
## [9,] 0.0000000 -0.5268316 1.38285390 -0.4875178 -0.03760863
## [10,] 0.0000000 0.0000000 -0.48751768 1.4902885 -0.41015602
## [11,] 0.0000000 0.0000000 -0.03760863 -0.4101560 1.25323238
```

```
# Visualize the estimated precision matrix
# Near-zero off-diagonal entries indicate conditional independence
image.plot(S_glasso$wi,
           main = "Graphical Lasso: Estimated Precision Matrix")
```



To illustrate graphical lasso, we simulate $p = 20$ variables with an AR(1) covariance structure. Although the covariance matrix is dense, its precision matrix is sparse, with conditional dependence occurring primarily between neighboring variables. Graphical lasso estimates this sparse precision matrix, and the resulting nonzero off-diagonal entries can be visualized as a network.

```
# Graphical Lasso application: simulate, fit, and visualize a network
set.seed(1)

# Load packages
library(glasso) # Graphical lasso for sparse precision matrix estimation
library(qgraph) # Network visualization

# ---- Simulate correlated data ----

# Specify the number of variables and observations
p <- 20
```

```

n <- 120

# Construct an AR(1) covariance matrix
# Correlation decreases with distance between variables
rho <- 0.6
Sigma <- rho ^ abs(outer(1:p, 1:p, "-"))

# Simulate n observations from N_p(0, Sigma)
X <- MASS::mvrnorm(n = n, mu = rep(0, p), Sigma = Sigma)

# Standardize each variable to mean 0 and variance 1
Xsc <- scale(X)

# Compute the sample covariance matrix
S <- cov(Xsc)

# ---- Estimate a sparse precision matrix using Graphical Lasso ----

# Set the penalty parameter
# Larger lambda produces a sparser estimated precision matrix
lambda <- 0.12

# Fit the Graphical Lasso
fit <- glasso(s = S, rho = lambda)

# Extract the estimated precision matrix
Theta_hat <- fit$wi

# Identify nonzero off-diagonal entries as network edges
Adj <- abs(Theta_hat) > 1e-6
diag(Adj) <- FALSE # Remove self-connections

# Use absolute precision-matrix entries as edge weights
W <- abs(Theta_hat)
diag(W) <- 0

# ---- Visualize the estimated conditional dependence network ----

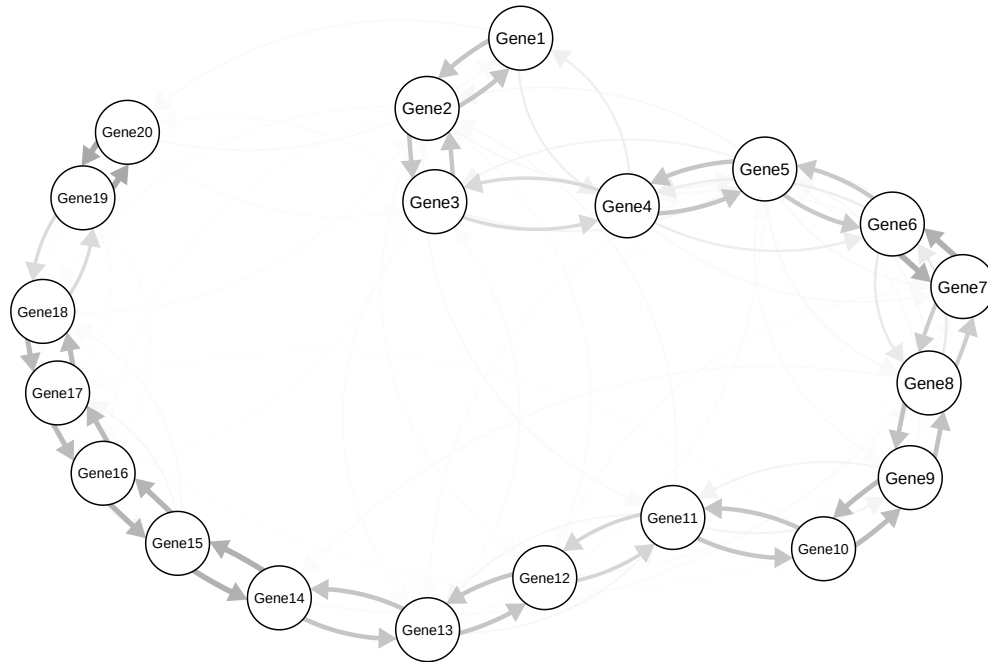
# Create variable labels
gene_labels <- paste0("Gene", 1:p)

# Plot the network
# Nodes represent variables; edges represent nonzero precision entries
qgraph(W,
  labels = gene_labels,
  layout = "spring",
  cut = 0,
  edge.color = "darkgray",
  asize = 4,
  vsize = 6,
  esize = 4,
  minimum = min(W[W > 0]),
  theme = "colorblind",

```

```
title = "Graphical Lasso Network (Nonzeros in Precision)"
```

Graphical Lasso Network (Nonzeros in Precision)



```
# ---- Summarize the estimated network ----  
  
# Count the number of unique edges  
num_edges <- sum(Adj[upper.tri(Adj)])  
  
cat("Nonzero off-diagonal entries (edges):", num_edges, "\n")
```

```
## Nonzero off-diagonal entries (edges): 50
```

Summary

This R Markdown covered key concepts and tools for covariance estimation including:

- Sample covariance and correlation
- Wishart distribution and inference
- Bootstrap and likelihood-based inference
- Regularized covariance estimation (shrinkage)
- Sparse precision estimation and motivation in high-dimensional settings